

Técnicas de Documentación y Validación

D&V de API





Contenidos

- API
- ¿Qué son y para qué sirven?
- OpenAPI 3.0
 - Documentación
 - Testeo
 - Deploy
- Testeo - Postman - cURL -Supertest



¿Qué es un API?

Mecanismos que permiten a dos (o más) componentes de software comunicarse entre sí.

Se establecen definiciones y protocolos

Un API (Application Program Interface) establece un contrato entre ambos componentes

La documentación es clave para la diseñar la estructura de solicitud/respuesta

Actualmente, las más utilizadas son las API Web (o API de servicios Web)



¿Qué tipos existen?

- SOAP: Cliente y servidor intercambian mensajes para proveer un acceso simple a objetos (XML)
- RPC: El cliente realiza la ejecución de una función de manera remota en el servidor.
- WebSocket: Cliente y servidor mantienen una comunicación bidireccional. El cliente puede enviar solicitudes, pero el servidor puede enviar datos al cliente. (JSON)
- REST: El cliente envía solicitudes al servidor, este las procesa y devuelve los resultados al cliente



¿Qué es un REST API?

- **REST:** Representational **S**tate **T**ransfer es un estilo arquitectónico y un enfoque de comunicación utilizado en el desarrollo de Web Services y APIs.
- **APIs RESTful:**
 - Simple
 - Escalable
 - Flexible
- **Usos:** Aplicaciones móviles, Web, IoT, arquitecturas de microservicios



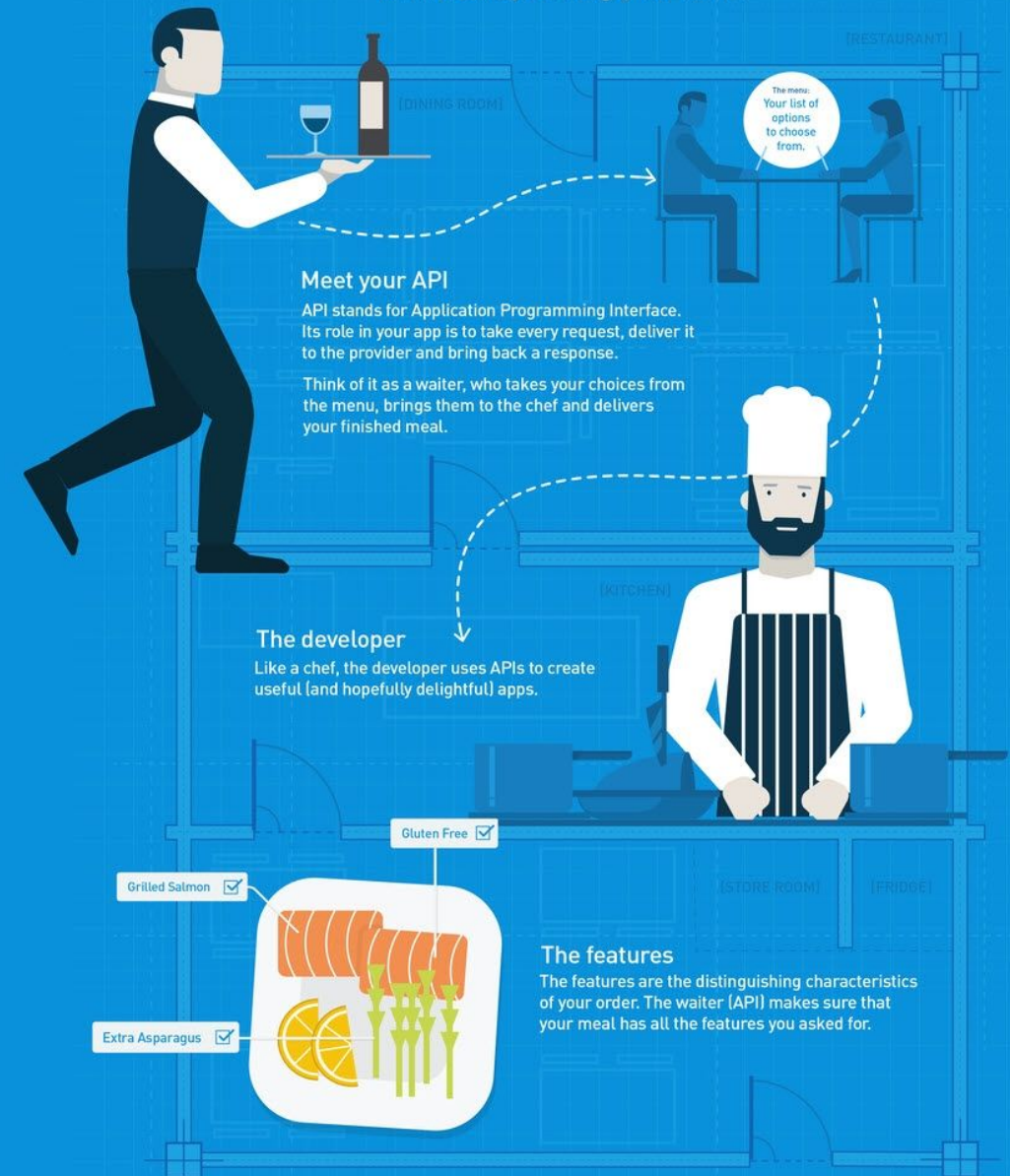
Características

- **Stateless:** La solicitud contiene toda la información para ser procesada, minimizando la necesidad de tener información para cada sesión
- **Basada en recursos:** Cada recurso es identificado por su URI y accedido mediante métodos estándar HTTP
- **Interfaz uniforme:** Permite a los clientes interactuar utilizando un conjunto estandarizado de métodos y formatos (Fácil de usar, fácil de mantener)
- **Cacheable:** Respuestas comunes permiten almacenarse de manera intermedia mejorando el rendimiento

El restaurante sin mesero ¿Qué hacemos?

THE ABCs OF APIs

What are APIs and what's the big deal about them?
We'll break it down for you in an easy-to-digest way
that'll leave you hungry for more.

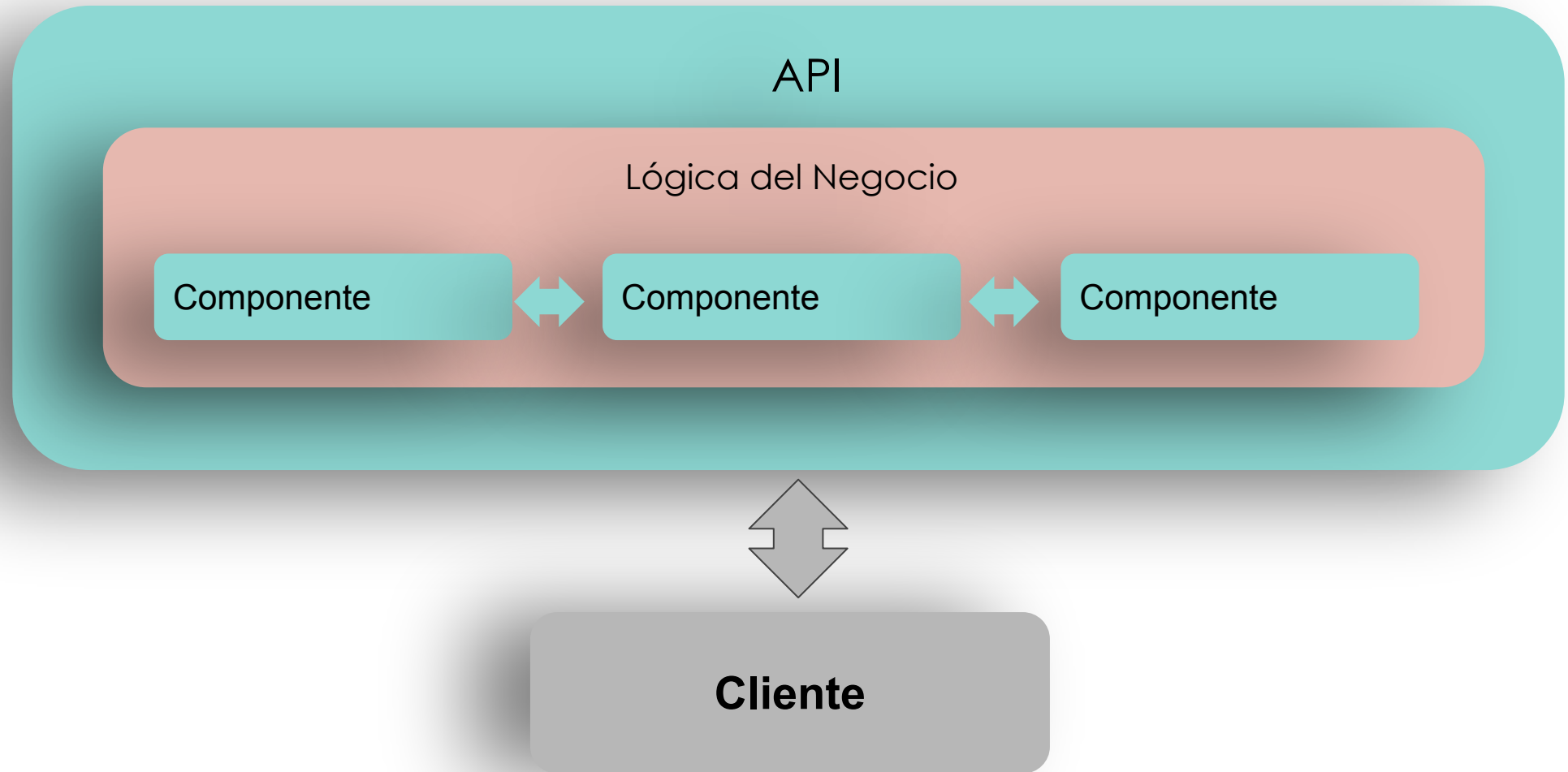


¿Cómo funciona?

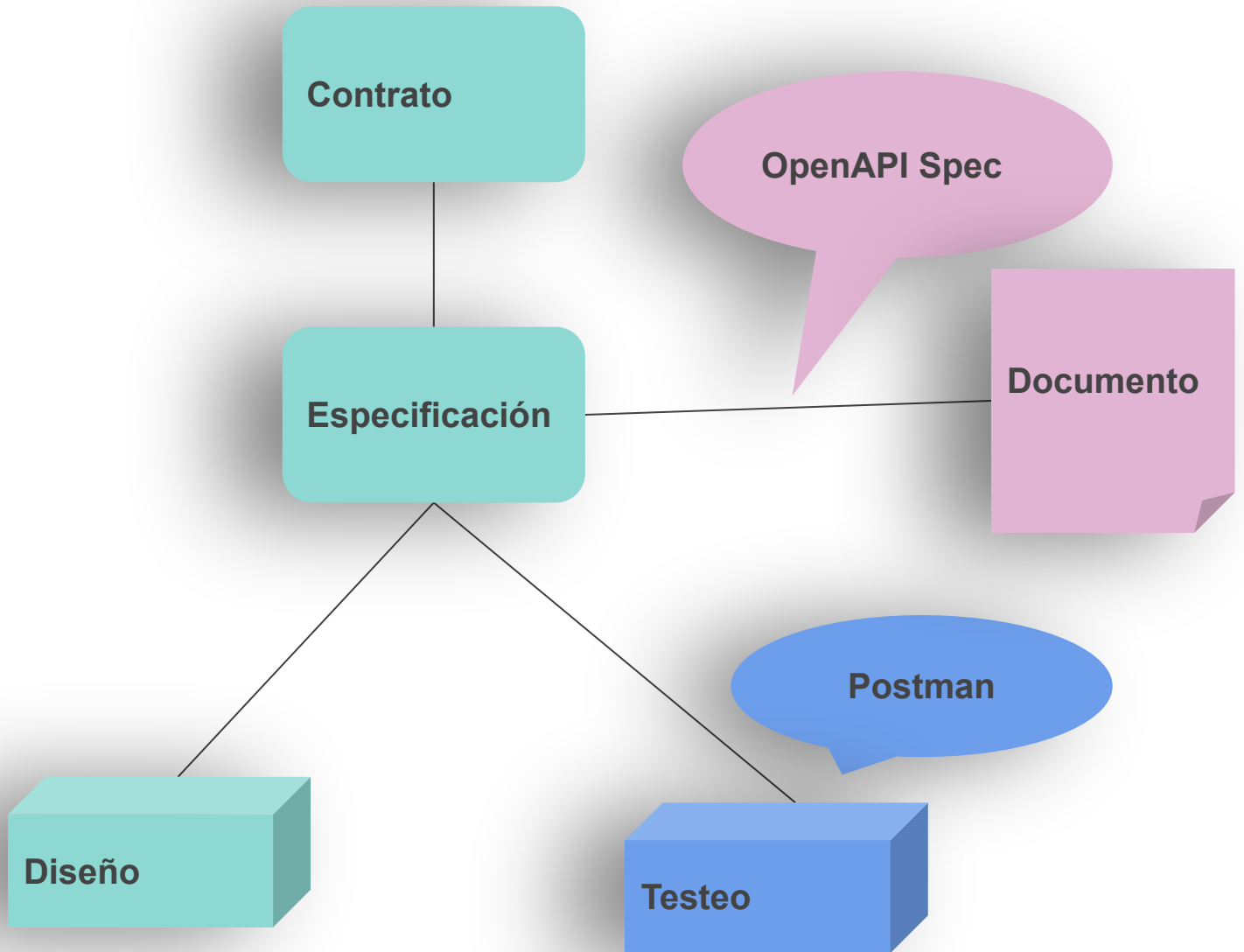
Un API toma un requerimiento desde un dispositivo y captura la respuesta de un servidor. Los request (o solicitudes o métodos o verbos) pueden ser:

SAFE METHODS NO ACTION ON SERVER	{	GET	HTTP/1.1 MUST IMPLEMENT THIS METHOD
		HEAD	INSPECT RESOURCE HEADERS
MESSAGE WITH BODY SEND DATA TO SERVER	{	PUT	DEPOSIT DATA ON SERVER — INVERSE OF GET
		POST	SEND INPUT DATA FOR PROCESSING
		PATCH	PARTIALLY MODIFY A RESOURCE
		TRACE	ECHO BACK RECEIVED MESSAGE
		OPTIONS	SERVER CAPABILITIES
		DELETE	DELETE A RESOURCE — NOT GUARANTEED

¿Que abordaremos?



¿Qué relación tienen la D&V de API?





Documentación de API

OpenAPI Spec 3.0

- Es un formato de código abierto para describir y documentar APIs. Desarrollada en 2010 por Reverb Technologies para mantener el diseño y documentación en perfecta sincronía.
- Se ha convertido en un estándar de-facto y es usada por millones de desarrolladores y organizaciones.
- La versión actual, OpenAPI 3.0 permite definiciones escritas en YAML y JSON.
- En adelante, se utilizará YAML ya que es más simple de leer y escribir.



Un ejemplo

openapi: 3.0.0

info:

version: 1.0.0

title: API Clubes

description: API para obtener información sobre socios de una entidad Social y Deportiva

paths:

/list:

get:

description: Retorna una lista de socios

responses:

'200':

description: Lista de socios fue obtenida



Forma

- Meta - Información
- Paths de los ítems
 - Parámetros
 - Cuerpo de consulta
 - Respuestas
- Componentes reusables
 - Esquemas
 - Parámetros
 - Respuestas
 - Otros componentes



openapi: 3.0.0

info:

version: 1.0.0

title: Simple API Socios

description: API de muestra para evaluar openapi 3.0

servers:

- url: <https://www.tdyv.com.ar/api>

Basic authentication

components:

securitySchemes:

BasicAuth:

type: http

scheme: basic

security:

- BasicAuth: []

paths: {}

Meta-información



Paths

```
# -----esto es un comentario
```

```
paths:
```

```
  /socios:
```

```
    get:
```

```
      description: Retorna una lista de socios
```

```
# ---- Esto es otro comentario
```

```
  /actividades:
```

```
    get:
```

```
      description: Retorna una lista de actividades
```



responses:

'200':

description: Se obtuvo la lista de socios

content:

application/json:

schema:

type: array

items:

type: object

required:

- idsocio

properties:

nombre:

type: string

apellido:

type: string

edad:

type: integer

dni:

type: string

responses:

--- continuación del bloque de response

'400':

description: Consulta inválida

content:

application/json:

schema:

type: object

properties:

mensaje:

type: string

Respuestas (Responses)

Parameters - Contenido

Se deben especificar:

- Nombre (name)
- Ubicación (location (in))
- Tipo del dato (definido en schema o content)
- Otros:
 - Descripción (description)
 - Obligatorio (required)

```
paths:
  /socios/{idSocio}:
    get:
      summary: Obtiene socio por su id
      parameters:
        - in: path
          name: idSocio
          schema:
            type: integer
            required: true
            description: ID numerico del socio a
              obtener
```



Parameters - Ubicación

Ubicaciones de los parámetros:

De ruta: /socios/1234

- De consulta: /socios?idSocio=1234
- De encabezado: X-idSocio: 1234
- De cookie: Cookie: idSocio=1234;



Parámetros - Parámetros de Ruta

- Son variables en la url utilizados para identificar un elemento de la colección.
- Se indican con { }

```
paths:  
  /socios/{idSocio}:  
    get:  
      parameters:  
        - in: path  
          name: idSocio  
          required: true  
          schema:  
            type: integer  
            minimum: 1  
          description: El ID a buscar
```

Parámetros - Parámetros de Consulta

- Se utilizan para flexibilizar la consulta concatenando criterios
- Aparecen al final de la url luego de un (?)
- Los criterios se separan con (&)

```
paths:
  /socios?contiene=Carl&edadLimite=30
  get:
    parameters:
      - in: query
        name: contiene
        schema:
          type: string
        description: contenido parcial
      - in: query
        name: edadLimite
        schema:
          type: integer
        description: Edad límite a buscar
```

Parámetros - Schema vs Content

```
paths:
  /socios
    get:
      parameters:
        - in: query
          name: idSocio
      schema:
        type: array
        items:
          type: int
      style: form
      explode: false
# /socios?idSocio={1,3,5,8}
```

```
paths:
  /socios
    get:
      parameters:
        - in: query
          name: filter
      content:
        application/json:
          schema:
            type: object
            properties:
              nombre:
                type: string
              edad:
                type: int
# ...?filter={"nombre":"Carlos","edad":25}
```



Parameters - Modificadores

```
parameters:  
  - in: query  
    name: edad  
    schema:  
      type: integer  
      minimum: 0  
      maximum: 100  
      default: 16  
    required: false  
    description: Edad del socio
```

```
parameters:  
  - in: query  
    name: plan  
    schema:  
      type: string  
    enum:  
      - estandar  
      - familiar  
      - premium
```

```
parameters:  
  - in: query  
    name: idActividad  
    schema:  
      type: integer  
      format: int32  
      nullable: true
```

```
parameters:  
  - in: query  
    name: edad  
    schema:  
      type: integer  
      minimum: 1  
      example: 20
```

Consulta - Request body

Usado en métodos:

- POST
- PUT
- PATCH

Generalmente es necesario
describir un cuerpo de la consulta

Se describe en el objeto

- requestBody

```
paths:
  /socios/:
    post:
      description: da de alta un socio
      requestBody:
        required: true
        content:
          application/json
          schema:
            type: object
            required:
              - nombre
              - apellido
              - edad
            properties:
              nombre:
                type: string
              apellido:
                type: string
```



Componentes reusables

En general, una API describe el intercambio de datos de un modelo

Un objeto participa en más de un endpoint!!! Múltiples definiciones

Componentes reusables

- Schemas (data models)
- Parameters
- Request bodies
- Responses
- Response headers
- Examples
- Links
- Callbacks

Componentes reusables - Schemas

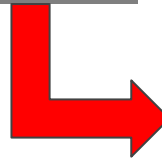
```
responses:  
  '200':  
    description: Se retorna la lista de socios  
    content:  
      application/json:  
        schema:  
          type: array  
          items:  
            $ref: '#/components/schemas/Socio'
```

```
components:  
  schemas:  
    Socio:  
      type: object  
      required:  
        - nombre  
        - apellido  
      properties:  
        nombre:  
          type: string  
        apellido:  
          type: string  
        edad:  
          type: integer
```

Se obtienen
especificaciones más
cortas, coherentes y sin
redundancia

Componentes reusables - Parámetros

```
paths:  
  /Socios:  
    get:  
      description: Retorna uno o mas socios  
      parameters:  
        - $ref: '#/components/parameters/FiltroSocio'
```



```
components:  
  securitySchemes:  
    ...  
  schemas:  
    ...  
  parameters:  
    FiltroSocio:  
      name: idSocio  
      in: query  
      description: Socio a buscar  
      schema:  
        type: integer
```



Testeo de API

Se puede realizar mediante:

- Software ad-hoc (ej. JUnit)
- Swagger
- Scripts (Curl)
- Herramientas (ej. Postman)



cURL



Está compuesto de:

Una biblioteca (libcURL)

Un CLI (cURL)

Protocolos: FTP, FTPS, HTTP, HTTPS, TFTP, SCP, SFTP, Telnet, DICT, FILE y LDAP

Software libre: Licencia MIT

```
curl  
"https://api.open-meteo.com/v1/forecast?latitude=-37.322&longitude=-59.13&hourly=temperature_2m,relativehumidit  
y_2m"|jq  
curl -X POST http://www.tdyv.com/login/ -d 'username=pepito&password=123456'
```

Swagger

Además de documentar el API es posible generar las pruebas manuales desde la documentación

httpbin.org^{0.9.2}

[Base URL: httpbin.org/]

A simple HTTP Request & Response Service.

Run locally: `$ docker run -p 80:80 kennethreitz/httpbin`

[the developer - Website](#)

[Send email to the developer](#)

Schemes

HTTPS

HTTP Methods Testing different HTTP verbs

DELETE /delete The request's DELETE parameters.

GET /get The request's query parameters.

PATCH /patch The request's PATCH parameters.

POST /post The request's POST parameters.

PUT /put The request's PUT parameters.



GET /get The request's query parameters.

Parameters Cancel

No parameters

Execute Clear

Responses Response content type: application/json

Curl

```
curl -X GET "https://httpbin.org/get" -H "accept: application/json"
```

Request URL

https://httpbin.org/get

Server response

Code	Details
200	Response body <pre>{ "args": {}, "headers": { "Accept": "application/json", "Accept-Encoding": "gzip, deflate, br", "Accept-Language": "es-ES,es;q=0.9,en;q=0.8", "Host": "httpbin.org", "Referer": "https://httpbin.org/", "Sec-Ch-Ua": "\"Google Chrome\";v=\"87\"; \"Not;A Brand\";v=\"99\"", "Sec-Ch-Ua-Mobile": "0", "Sec-Ch-Ua-Platform": "Linux", "Sec-Fetch-Dest": "empty", "Sec-Fetch-Mode": "cors", "Sec-Fetch-Site": "same-origin", "User-Agent": "Mozilla/5.0 (X11; Linux x86_64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/97.0.4692.88 Safari/537.36", "X-Amzn-Trace-Id": "Root=1-61759d38-20f3a5e50317dbc4789dead5" }, "origin": "190.17.59.137", "url": "https://httpbin.org/get" }</pre> Download

Response headers

```
access-control-allow-credentials: true
access-control-allow-origin: *
content-length: 727
content-type: application/json
date: Sun, 24 Oct 2021 17:51:44 GMT
server: gunicorn/19.9.0
```

Responses

Code	Description
200	The request's query parameters.



Postman



- Es una herramienta escalable para testeo de API
- Fácil integración en entornos de IC/DC (CI/CD)
- Permite uso de colecciones (equivalentes a test suites)
- Permite definición de entornos
- Permite la creación de tests
- Salidas de Debug

Postman - Tests



Permite describir en código JavaScript los tests

Provee un conjunto de los snippets más comunes para ser usados fácilmente

```
tests["Status code is 200"] = responseCode.code === 200;
if(responseCode.code === 200) {
  var jsonData = JSON.parse(responseBody);
  tests["Hace calor"] = jsonData.current_weather.temperature >
  15;
}
```